

Task 3: Optimisation Project

Group Chenonceau: Particle Swarm and Simulated Annealing Optimisation Project (constrained and unconstrained)



Group Leader: John Hoarau

Group Members:

Martin Harambat, Pierre Bédrune, Clémence-Philomène Hinot

Prepared for December 14, 2025

Contents

1	Introduction	3
2	Methodology and Algorithms	4
2.1	Method 1: Particle Swarm Optimisation (PSO)	4
2.1.1	Single-objective algorithm	4
2.2	Method 2: Simulated Annealing (SA)	6
3	Problem 1: Two-Bar Plane Truss Design	9
3.1	Problem description	9
3.2	Methodology	11
3.2.1	General explanation	11
3.2.2	Penalty function	11
3.3	Results	11
3.3.1	PSO simulation	11
3.3.2	SA simulation	14
3.4	Discussion	18
4	Problem 2: Sensor Calibration	20
4.1	Problem description	20
4.2	Cost function	20
4.3	Methodology	20
4.4	Settings	21
4.5	Result	21
4.6	Work discussion	22
5	Problem 3: Gear Train Design	24
5.1	Problem description	24
5.2	Cost function	24
5.3	Settings	25
5.4	Results	25
5.4.1	PSO simulation	25
5.4.2	SA simulation	26

5.5	Discussion	27
6	General Conclusion	29
6.1	Performance Comparison	29
6.2	Hyperparameter Sensitivity and Robustness	30
7	References	32
8	Appendix	33
8.1	Multi-objective Particle Swarm Optimisation choice	33
8.2	Python code	33
8.2.1	Two-Bar Plane Truss Problem Definition	33
8.2.2	PSO Simulation Script (Truss)	40
8.2.3	Steinhart-Hart Problem Definition	59
8.2.4	Simulated Annealing Class	65
8.2.5	Gear Train Problem and Runner	74

1 Introduction

This report investigates unconstrained and constrained nonlinear optimisation techniques. Specifically, it compares the performance of **Particle Swarm Optimisation (PSO)** and **Simulated Annealing (SA)** [1]. These algorithms are applied to three distinct engineering problems:

1. **Two-Bar Plane Truss Design** (Constrained, Multi-objective)
2. **Sensor Calibration** (Unconstrained, Curve fitting)
3. **Gear Train Design** (Constrained, Discrete/Integer optimization)

Statement on the Use of Generative AI During the preparation of this work, we, as the authors, used ChatGPT (OpenAI) [2] and Claude AI (Anthropic) [3] to assist with code efficiency optimisation and debugging for the Particle Swarm Optimisation and Simulated Annealing scripts. After using this tool, we reviewed and edited the content as needed and took full responsibility for the final report's content and the code's functionality.

2 Methodology and Algorithms

2.1 Method 1: Particle Swarm Optimisation (PSO)

The Particle Swarm Optimisation (PSO) is a stochastic optimisation technique inspired by the social behaviour of birds. A population of candidate solutions moves through the domain. Through several time steps, each particle adjusts its position based on its personal best and on the global best. This allows the algorithm to converge to a solution that matches a criterion [4].

2.1.1 Single-objective algorithm

For single-objective problems (like the Gear Train Design), the standard PSO is used to minimise a single error function.

- **Continuous Movement:** Particles move in a continuous space according to standard velocity and position update rules.
- **Integer Rounding (Discrete Adaptation):** For the Gear Train Design problem, which requires an integer number of teeth, the particle positions are rounded to the nearest integer to satisfy the discrete constraint.
- **Convergence Tracking:** The best error (difference from the target ratio) is stored at each iteration to generate a convergence curve.

2.1.2 Multi-objective algorithm

Description

The multi-objective PSO finds Pareto-optimal solutions using a selection based on dominance and diversity [5]. It offers advantages such as ease of implementation, few parameters to

tune, and good applicability to nonlinear problems, but also limitations such as possible early convergence, sensitivity to parameter settings, and no guarantee of global optimality [4].

The MOPSO-CD (Multi-Objective Particle Swarm Optimisation with Crowding Distance) is an advanced extension of the PSO. This particular algorithm was chosen from a list of alternative solutions based on specific criteria (section 8.1). Its key features are:

- Crowding distance-based leader selection,
- External archive management,
- Tournament selection,
- Enhanced exploration.

Performance metrics

Metrics are used to measure the algorithm's performance. We have selected some of the most common criteria:

- The best value per objective, which tracks the minimum value achieved for each objective function at every generation,
- The Pareto front size evolution, which tracks how many solutions are on the Pareto front at each generation,
- The normalised convergence, which compares both objectives normalised to $[0, 1]$.

2.2 Method 2: Simulated Annealing (SA)

Problem description

Simulated Annealing is an optimisation method inspired by the annealing process in metallurgy [6]. The algorithm explores the search space and accepts neighbouring solutions. In this algorithm, ameliorations are always accepted, and degradations are accepted with a probability. The probability decreases stepwise (decreasing temperature). At the beginning, this acceptance mechanism helps to escape local minima. Afterwards, the algorithm converges to a global minimum [4].

Cost function

In this method, we use this function to decide if a worst solution can be accepted:

$$P(\text{acceptance}) = e^{-\frac{E_{new} - E_{actual}}{T}} \quad (1)$$

Where:

- E_{actual} : Actual energy
- E_{new} : The new worst energy
- $P(\text{acceptance})$: the probability to accept E_{new}
- T : temperature parameter

Every iteration the temperature T decreases by the cooling_rate:

$$T = T \times \text{cooling_rate} \quad (2)$$

Multi-Objective Strategy (Weight Scalarization)

Unlike MOPSO, the Simulated Annealing program is inherently a mono-objective algorithm. To solve the bi-objective Truss problem, a scalarization approach is used to transform the problem into a single objective function.

- **Scalarization:** A weighted sum combines the two objectives (Weight and Displacement) into a single scalar Energy function.
- **Normalisation :** A scaling factor (Scale = 10000) is applied to the displacement term to normalise both objectives to comparable orders of magnitude.
- **Weight Distribution:** To approximate a Pareto front, 200 independent runs are performed with the weight w_1 uniformly distributed between 0.005 and 0.995 (where $w_2 = 1 - w_1$). This allows the algorithm to explore different trade-offs, from minimising weight ($w_1 \approx 1$) to minimising displacement ($w_1 \approx 0$).

Performance metrics

To evaluate the performance of the SA algorithm, the following metrics were tracked:

- **Convergence profiles:** Analysis of the best energy and current energy evolution over iterations to assess stability and exploration.
- **Pareto Front Approximation:** The collection of non-dominated solutions found across the 200 independent runs is plotted to visualise the trade-off curve (the "Banana shape").
- **Decision Space Analysis:** Mapping the decision variables (x_1, x_2) of the final solutions to identify distinct regions corresponding to different trade-off preferences.

Application to Gear Train Design (Discrete Problem)

For the discrete gear train problem, the SA algorithm explores the search space by randomly perturbing the current solution (integer number of teeth).

- **Exploration:** High temperatures allow for wide exploration, accepting worse solutions with higher probability.
- **Exploitation:** As temperature decreases, the algorithm converges around a good solution, calculating the probability of acceptance based on the energy difference.
- **Constraint Handling:** The integer constraints are inherent to the neighbor generation process.

Settings

It is important to consider:

- *initial_temp*: the temperature at the beginning
- *cooling_rate*: The speed of the freezing
- *min_temp*: The temperature where we stop the process

3 Problem 1: Two-Bar Plane Truss Design

2.1.2 Multi-Objective algorithm

3.1 Problem description

The Two-Bar Truss problem is composed of two diagonal bars (of aluminium here) connected at a node and fixed at two upper points (Figure 1). On the central node, we applied a vertical strength. This force represents the external load that the structure must support. We look for the vertical displacement of the node.

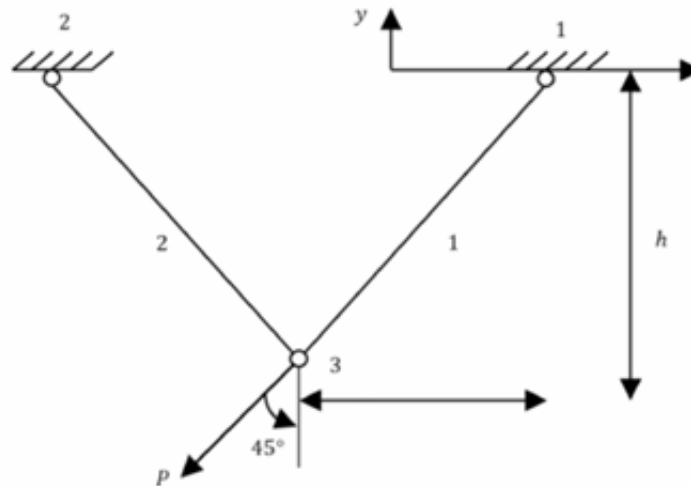


Figure 1: Representation of the two-bar plane truss problem [7]

There are two design variables: the positions of joints 1 and 2. They are defined by the two equations: f_1 , the structural weight truss, and f_2 , the displacement of joint 3. The conflict between minimising weight and minimising displacement is a classic example of multi-objective optimisation [5].

$$f_1(x) = 2\rho h x_2 \sqrt{(1 + x_1^2)} \quad (3)$$

$$f_2(x) = \frac{Ph(1 + x_1^2)^{1.5} \sqrt{(1 + x_1^4)}}{2\sqrt{2}Ex_1^2x_2} \quad (4)$$

These functions are constrained by two stress constraint functions g_1 (tension) and g_2 (compression).

$$g_1(x) = \frac{P(1 + x_1)\sqrt{1 + x_1^2}}{2\sqrt{2}x_1x_2} - \sigma_0 \leq 0 \quad (5)$$

$$g_2(x) = \frac{P(-x_1 + 1)\sqrt{1 + x_1^2}}{2\sqrt{2}x_1x_2} - \sigma_0 \leq 0 \quad (6)$$

The variables x_1 and x_2 are calculated from the following formulas: $x_1 = \frac{x}{h}$ and $x_2 = \frac{A}{A_{min}}$
 $0.1 \leq x_1 \leq 2.25$ and $0.5 \leq x_2 \leq 2.5$

If the x_1 variable is connected to the physical sense of the truss geometry, the position boundaries are: $x_{min} = 0.1 * h = 0.254m$ and $x_{max} = 2.25 * h = 5.72m$

The parameters are converted to International System units.

- $\rho = 0.283lb/in^3 = 7833kg/m^3$;
- $h = 100 \text{ in} = 2.54 \text{ m}$;
- $P = 10^4lb = 44482.2N$;
- $E = 3 \cdot 10^7lb/in^2 = 2.07e11Pa$;
- $\sigma_0 = 2 \cdot 10^4lb/in^2 = 1.38e8Pa$;
- $A_{min} = 1in^2 = 6.5416e - 4m^2$.

3.2 Methodology

3.2.1 General explanation

The problem is defined by a class containing four functions: two for the objectives and two for the constraints.

3.2.2 Penalty function

The Pymoo library already includes constraint handling and constraint-violation measures during selection. For the multi-objective problem, it uses the Pareto ranking. To study how the penalty strength affects convergence and feasibility, explicit penalty terms will be added [5].

$$F_{penalised} = f(x) + k * \sum (\max(0, g_i(x))^2) \quad (7)$$

With: k the penalty factor.

Considering this equation, a penalty will be applied when the maximum between 0 and g_i is g_i , so the constraint is violated. This is realised with the following loop, where the penalty factor is fixed to 10 billion. If the penalty factor is significant, the algorithm quickly avoids infeasible solutions. If the penalty factor is small, the algorithm may spend longer exploring infeasible regions.

3.3 Results

3.3.1 PSO simulation

Best solution

To obtain the best solution, the algorithm minimises the weight function and then the displacement function. Then it finds the best compromise that satisfies the objective functions

while satisfying the constraints.

Table 1: Summary of key solutions (PSO)

Solution type	x1 [normalised]	x2 [normalised]	Weight [kg]	Displacement [m]
Min weight	0.8245	0.5	16.6361	0.0084
Min displacement	1.8777	2.5	136.5345	0.0011
Compromise	1.2560	1.1667	48.0982	0.0029

All the possible values of the design variables are shown in the decision space diagram: each point represents a candidate solution. The colours indicate a gradient: dark colours for the lower objective values, which are better for minimisation, and lighter shades for the higher objective values, which are worse for minimisation.

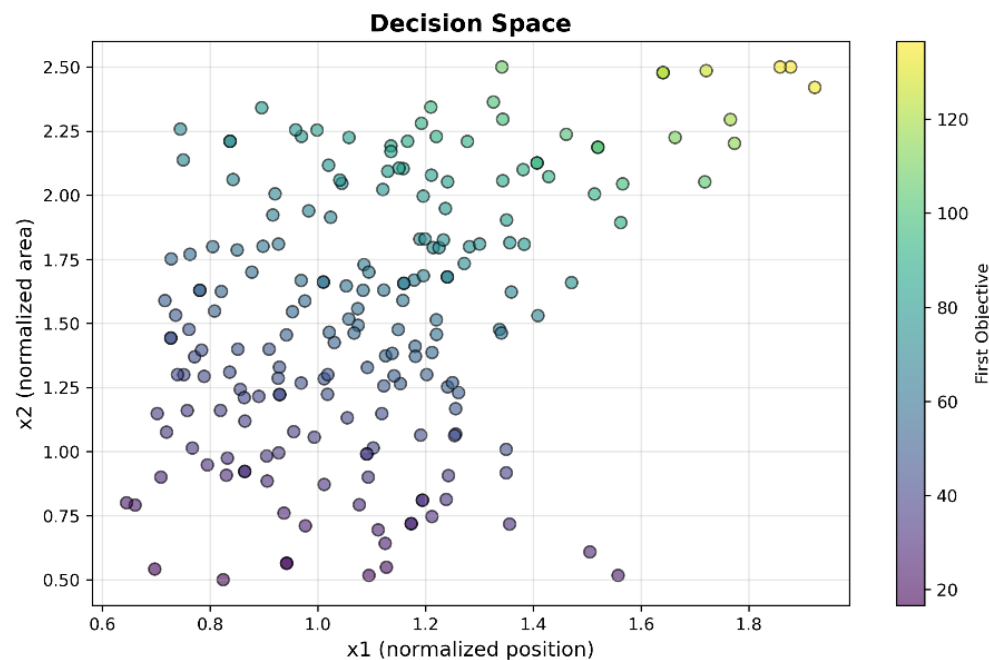


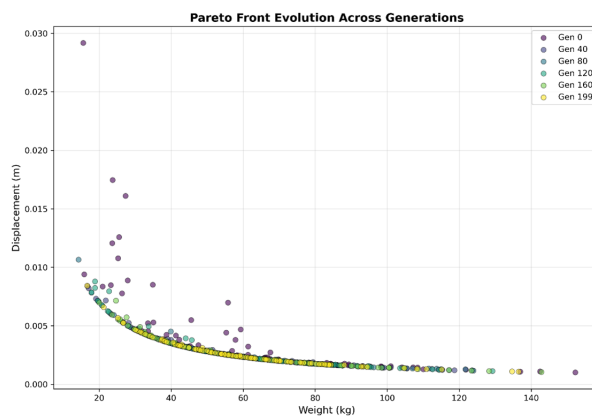
Figure 2: PSO Decision Space

Metrics

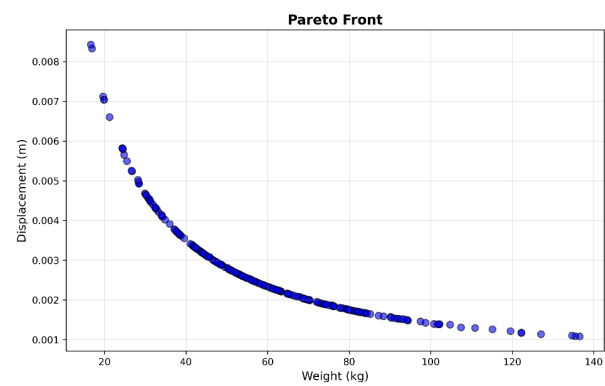
The Pareto front evolution across generations diagram shows that the first generations

are far away from the Pareto front, but through each time step, the solutions converge. This is possible thanks to the penalty function, which eliminates poor solutions. The visualisation helps identify regions of the decision space associated with the optimal solution.

The generation 199, or 200 time step, generates the Pareto front kept. The best solution is extracted from it.



(a) Pareto Front Evolution Across Generations



(b) PSO Pareto Front

Figure 3: Evolution of the Pareto Front and final result (PSO)

The convergence analyses the efficiency of the algorithm. The following diagrams describe the evolution of the best values at each generation.

- **Best value First objective:** The weight is minimised over time with an improvement of the solution. The curve decreases over the generations, so the minimum is stable.
- **Best value Second objective:** The algorithm finds solutions with smaller solutions through the generations. The curve converges around an optimal displacement.
- **Normalised convergence:** The objectives are normalised for comparison. The algorithm is simultaneously improving both objectives as the curves decrease.

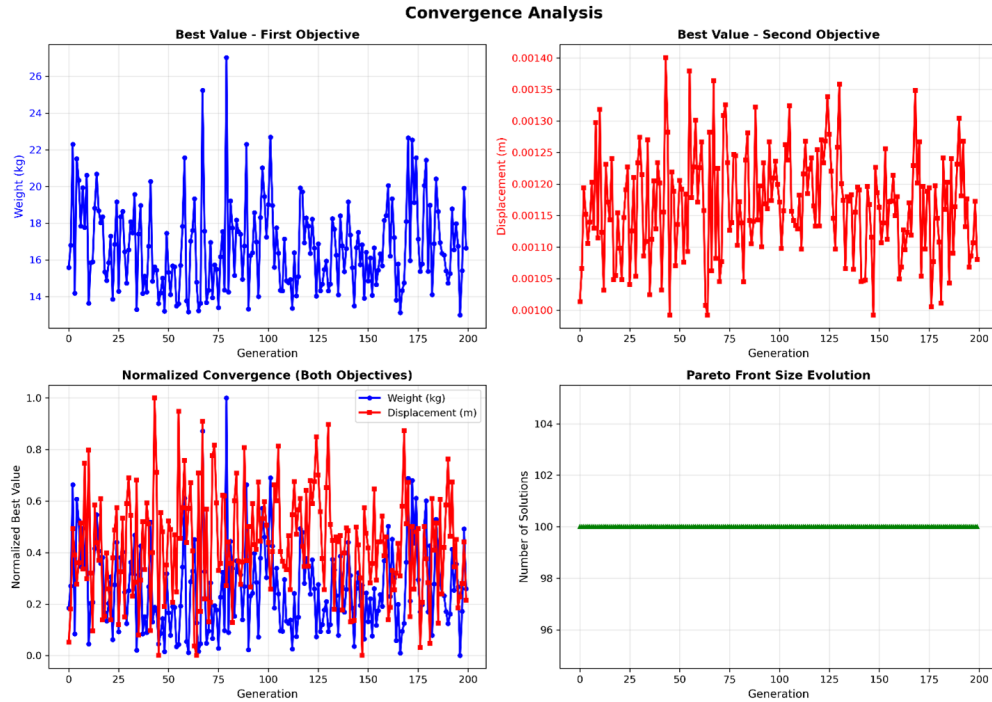


Figure 4: PSO Convergence Analysis

The Pareto front size evolution diagram shows the number of non-dominant solutions at each generation. Its constant size maintains diversity throughout optimisation. This is a good indicator to tell that the algorithm didn't lose diversity and didn't converge prematurely.

3.3.2 SA simulation

Weight Scalarization Strategy

The simulated annealing program is based on a mono-objective algorithm. We have a scalarization approach for transforming the problem (bi-objective) into a mono-objective function. To generate the Pareto front, 200 runs are made with the weight uniformly distributed between 0.005 and 0.995. A scale of 10000 is applied to the displacement for normalising both objectives to comparable orders of magnitude. The stress constraints are enforced in the

optimisation program via a penalty method, with the solution verified at the end. This double approach ensured that all Pareto fronts satisfied the structural constraints.

Scalar Energy function :

$$E(x) = w_1 \cdot f_1(x) + w_2 \cdot (scale_f2 \times f_2(x)) + \sum_{i=1,2} 10^6 \cdot \max(0, g_i(x)) \quad (8)$$

where :

- w_1, w_2 : scalar weight with $w_1 + w_2 = 1$ and $w_1 \in [0.005, 0.995]$
- $f_1(x)$: structural weight (kg)
- $f_2(x)$: displacement (m)
- $scale_f2 = 10000$: the objective normalize factor
- $g_i(x) \leq 0$: Stress constraints (tension g_1 and compression g_2)

Pareto Front

The Pareto front is the result of 200 runs of the Simulated Annealing program and exhibits an optimal distribution of solutions representing the trade-off between structural weight and node displacement. The valid solutions are comprised between: Weight: 14.39 kg to 213.64 kg Displacement: 0.848 mm to 8.382 mm The banana shape of the Pareto front saw the conflict between the structural optimisation: reduce the weight and increase the flexibility, and the displacement; otherwise, if we want rigidity, we need more material. We also found no single solution that simultaneously minimised both objectives.

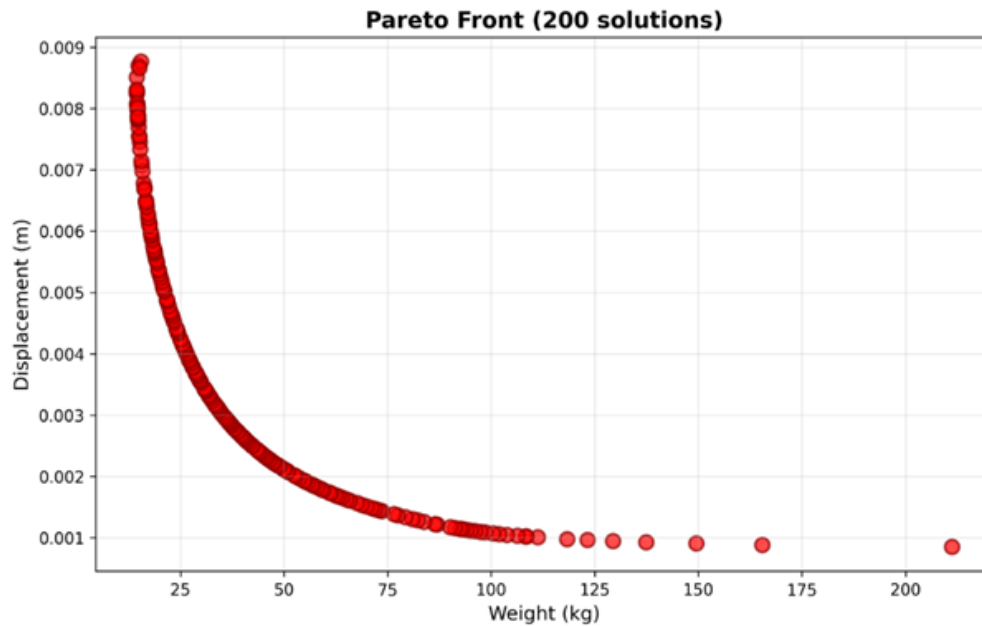


Figure 5: SA Pareto Front

Decision Space

The decision space distribution reveals a characteristic L shape. It is the result of the linear scalarization of the weight:

- Region 1 ($x_2 \in [0.5, 1.0]$, x_1 variable): light solution for the weight minimization ($w_1 \approx 1.0$)
- Region 2 ($x_2 \in [2.0, 2.5]$, x_1 variable): Minimize the node displacement ($w_1 \approx 0.0$)
- Space of transition ($x_2 \in [1.0, 2.0]$) : compromise solution

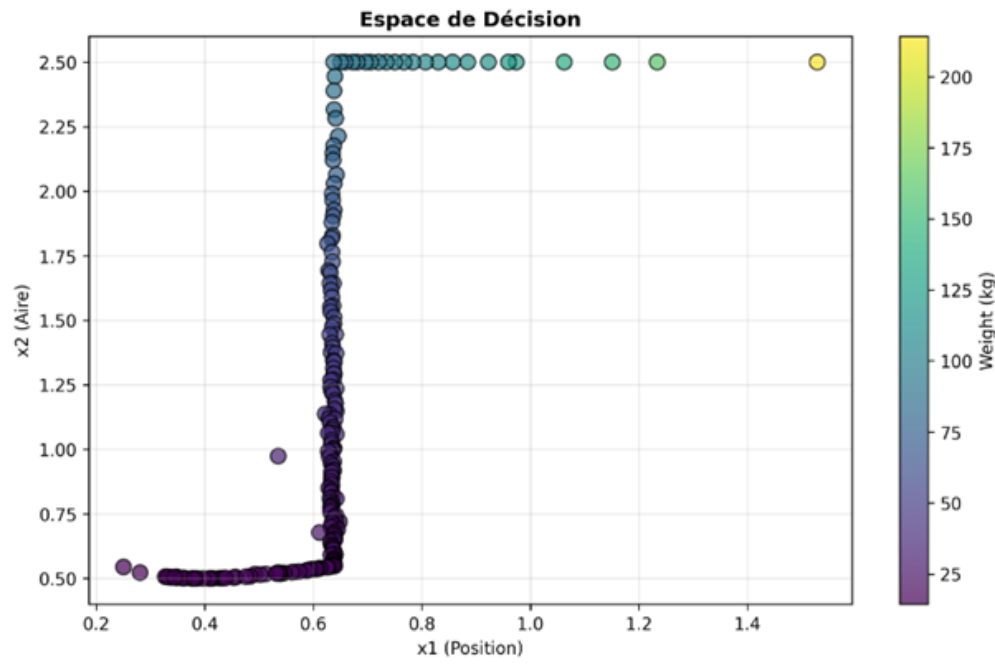


Figure 6: SA Decision Space

Convergence analysis

The convergence compartment of the 200 runs is represented on a distinct plot for each objective:

- Best weight evolution: Weight decreased to 200 kg by 14,39 kg. Significant improvement result after the first 50 runs. Stable execution after 150 runs that saw a complete exploration of the light region.
- Best displacement evolution: Speed convergence in 20 runs to the minimal value of 0,848 mm. We have a parabolic augmentation, and we start by optimising the displacement, so we have the best optimisation at the beginning and the worst after that.

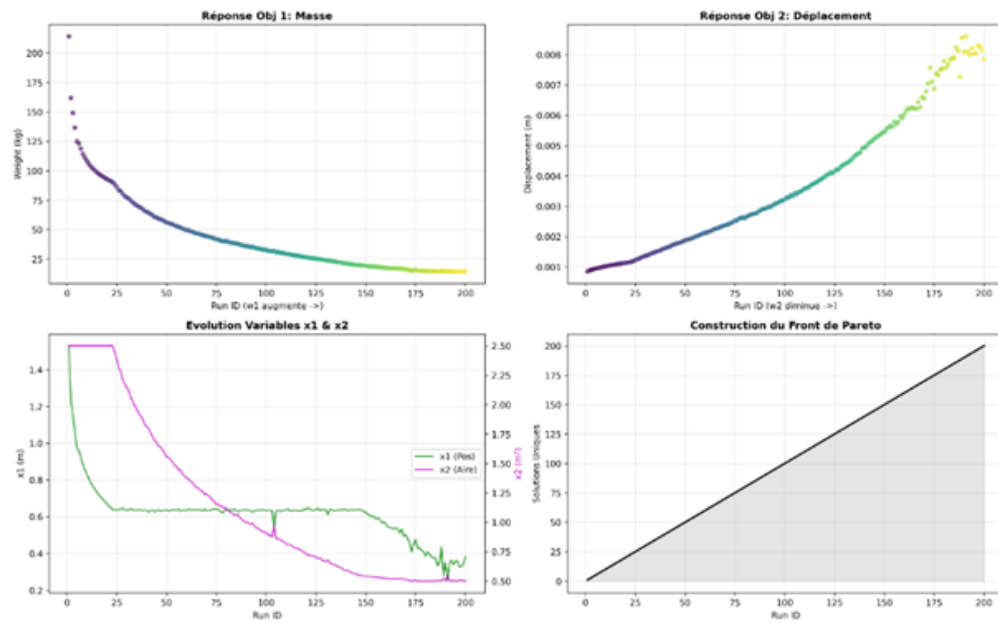


Figure 7: SA Convergence

3.4 Discussion

The best solutions found by the PSO and SA algorithms are almost similar. The differences between the two algorithms are explained by the proper operations they compute and the penalty treatment.

Table 2: Best solution comparison

Algorithm	x1 [normalised]	x2 [normalised]	Weight [kg]	Displacement [m]
PSO	1.2560	1.1667	48.0982	0.0029
SA	1.0429	1.0043	37.2536	0.0038

Table 3: Absolute differences between PSO and SA algorithms

Algorithm	x1 [normalised]	x2 [normalised]	Weight [kg]	Displacement [m]
Min	0.1822	0.037	0.2517	0.0003
Max	0.37	0	21.4903	0.0001
Compromise	0.2131	0.1624	10.8446	0.0009

These two algorithms are efficient for treating the Two-Bar Plane Truss problem. The differences can be seen with the complexity (number of actions to obtain the solution), the shape of the convergence curve, and the calculation cost.

Table 4: Performance comparison of PSO and SA algorithms

Algorithm	Complexity	Convergence	Calculation cost
SA with 200 runs	106 400	Smooth and monotonous	Low
SA with 200 generations	40 000	Oscillating	High

The SA algorithm compensates for its large number of steps with its simplicity of calculation. In contrast, the PSO algorithm is more computationally expensive. Penalties play an essential role in both algorithms. To conclude, the choice of algorithm depends on the study's requirements: its a balance between computational costs and algorithm performance.

4 Problem 2: Sensor Calibration

4.1 Problem description

The objective is to determine the Steinhart-Hart coefficients (A, B, C) that model the relationship between Resistance (R) and Temperature (T) for a 10K thermistor. Model Equation:

$$T_{model} = \frac{1}{A + B \ln(R) + C(\ln(R))^3} \quad (9)$$

Note: T is in Kelvin, R in Ohms.

4.2 Cost function

The optimisation goal is to minimise the Sum of Squared Errors (SSE) between the measured temperature and the model-predicted temperature:

$$SSE = \sum (T_{measured,i} - T_{model,i})^2 \quad (10)$$

4.3 Methodology

Simulated Annealing

A probabilistic technique for approximating the global optimum of a given function.

Particle Swarm Optimisation (PSO)

A computational method that optimises a problem by iteratively trying to improve a candidate solution based on a given measure of quality. It solves a problem by maintaining a population of candidate solutions, here dubbed particles, and moving them in the search space according to a simple mathematical formula that depends on their positions and velocities.

4.4 Settings

Settings SA

- Initial Temperature: 50.0
- Cooling Rate: 0.995
- Minimum Temperature: 1e-6

Settings PSO

Implementation utilises the pymoo library (PSO algorithm).

- **Parameters:**
 - Population size: 50
 - Generations: 500
 - Inertia weight (w): 0.7
 - Cognitive parameter (c_1): 1.5
 - Social parameter (c_2): 1.5
- max_velocity_rate: Limits the maximum velocity of particles.

4.5 Result

The optimal Steinhart-Hart coefficients found for the provided dataset are:

Table 5: Optimisation Results Comparison (Steinhart-Hart)

Algorithm	A	B	C	Final SSE	Function Calls	Execution Time (s)
Simulated Annealing	1.114×10^{-3}	2.367×10^{-4}	7.761×10^{-8}	0.4323	3,537	0.099
PSO	1.127×10^{-3}	2.345×10^{-4}	8.625×10^{-8}	0.0089	25,000	0.678

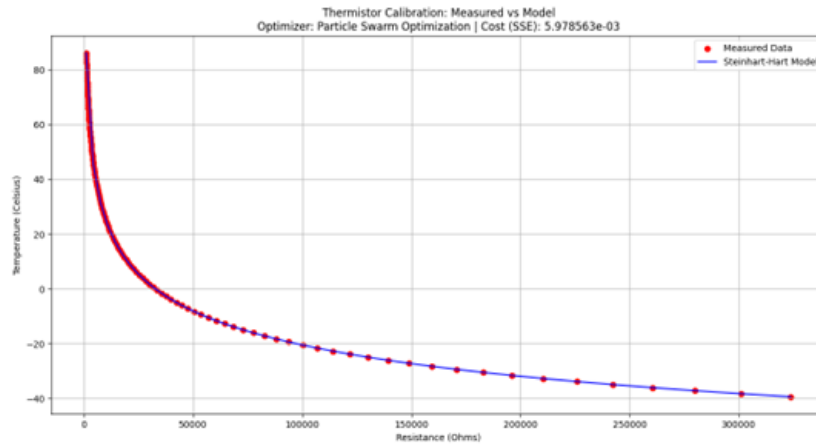
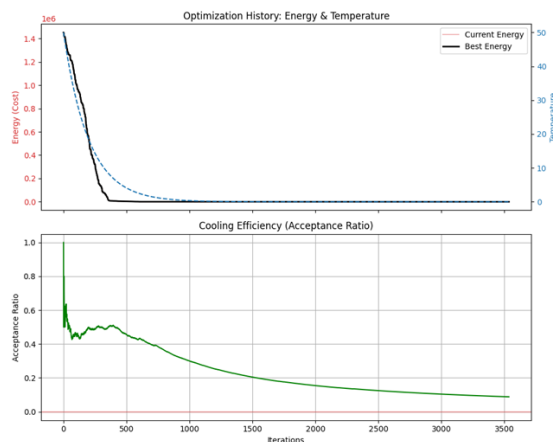
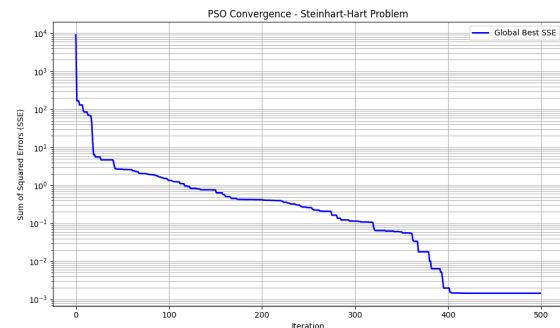


Figure 8: Steinhart-Hart Model Fit



(a) SA Convergence History



(b) PSO Convergence History

Figure 9: Convergence Comparison

4.6 Work discussion

Both algorithms successfully calibrated the thermistor model. PSO achieved a significantly lower final SSE (0.0089 versus 0.4323 for SA), indicating a more precise fit for this specific dataset and configuration. However, SA was roughly $6.8\times$ faster (0.099s vs 0.678s) and used significantly fewer function calls (3,537 vs 25,000). This illustrates the classic trade-

off: PSO's population-based search provided a better global optimum at the cost of higher computational expense. In contrast, SA provided a "good enough" solution very quickly.

5 Problem 3: Gear Train Design

5.1 Problem description

The objective of this exercise is to choose a gear train design that achieves a specific transmission ratio. We would define the gear train with four gears with the numbers of teeth N_1, N_2, N_3, N_4 . The target transmission ratio is:

$$R = \frac{N_2 \cdot N_4}{N_1 \cdot N_3} \quad (11)$$

(Note: Standard formulation is usually $\frac{\text{product driven}}{\text{product drivers}}$ or similar. The equation in text is $R = (N_2 N_4)/(N_1 N_3)$ or $f(N) = (N_1 N_3)/(N_2 N_4)$. I will use the cost function definition for consistency: $Target \approx 1/6.931 \approx 0.144$).

Here, the target transmission ratio will be randomly chosen: $R_{target} = 1/6.931 \approx 0.1442793$. And each gear must have an integer number of teeth between 12 and 60. The problem is complex because the search space is discrete and nonlinear, making classical optimisation infeasible.

5.2 Cost function

The cost function measures the error between the achieved ratio and the target ratio:

$$f(N_1, N_2, N_3, N_4) = \left| \frac{N_1 \times N_3}{N_2 \times N_4} - R_{target} \right| \quad (12)$$

A good solution is one where this error approaches zero.

5.3 Settings

PSO Settings

- Population size: 40
- Iterations: 300
- $w = 0.7, C_1 = 1.5, C_2 = 1.5$
- Integer rounding of particle positions
- Recorded global best error per iteration (for convergence curve)

SA Settings

- Initial temperature: 1000
- Cooling rate: 0.90
- Minimum temperature: 0.1
- Random seed: 42
- Recorded: Current energy / Best energy (for convergence curve)

5.4 Results

5.4.1 PSO simulation

Best Solution and Convergence

The optimal solution found by the Particle Swarm Optimisation is extremely accurate, with an error on the order of 10^{-5} , meaning it matches the target ratio almost exactly.

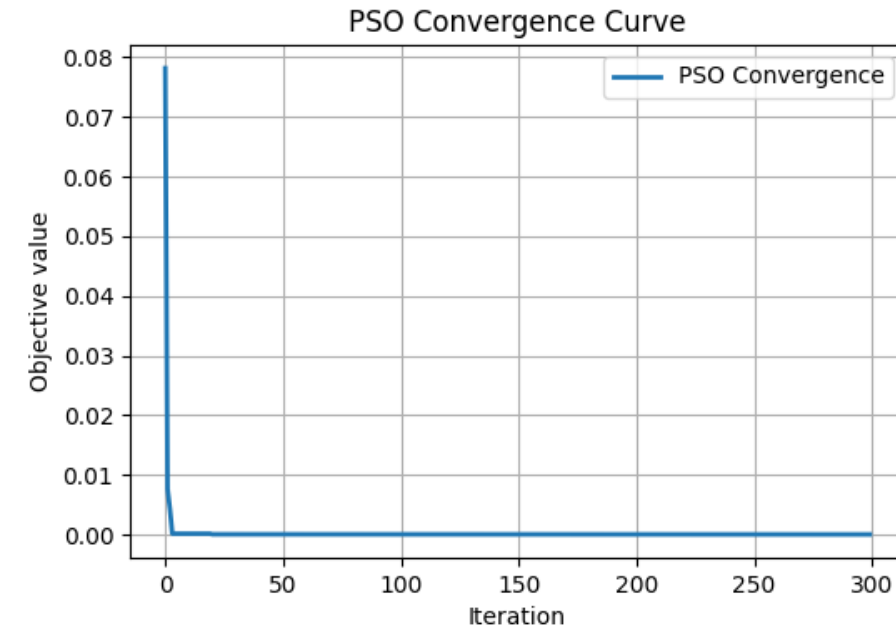


Figure 10: PSO Convergence curve

The graph shows that PSO converges extremely quickly within ≈ 10 iterations, then stabilises at a minimal error.

5.4.2 SA simulation

Best Solution and Convergence

The result found by the Simulated Annealing (SA) clearly fails on this problem. The final ratio is far from the target, with a considerable error.

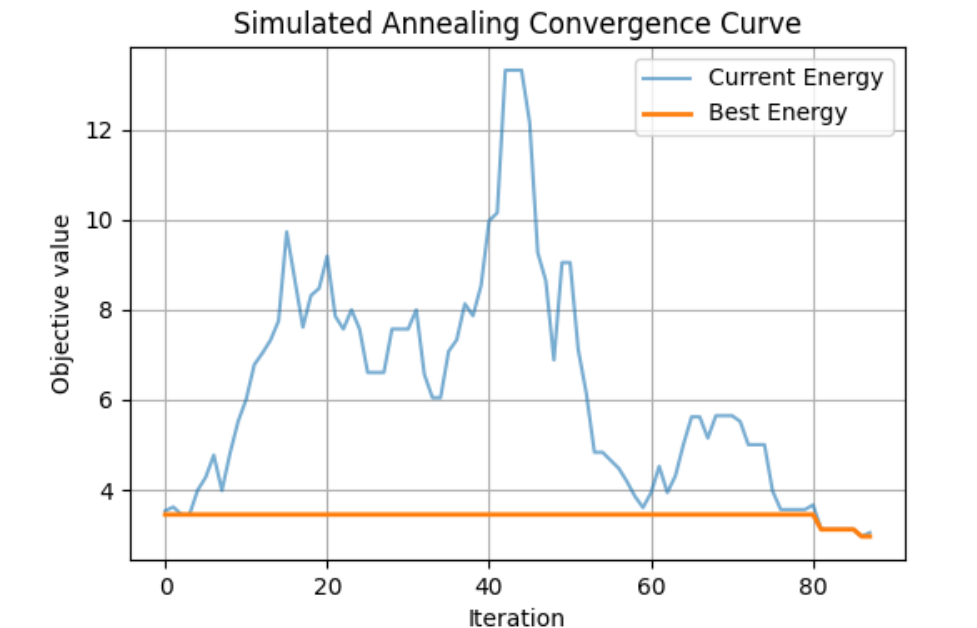


Figure 11: SA Convergence Curve

The plot shows:

- Highly unstable exploration, current energy jumps up and down.
- Best energy stays flat because the algorithm never discovers a good solution.
- Cooling happens too fast to explore thoroughly.

5.5 Discussion

Why PSO succeeds

- PSO is particularly effective in structured continuous search spaces, even with integer rounding.
- The particles rapidly converge to the correct ratio because the cost function is smooth.

- Global and personal best mechanisms efficiently guide the search.

Why SA fails

- The gear train problem is very sensitive: small changes in N_i produce large jumps in the ratio.
- SA explores randomly and relies on thermal acceptance, which:
 - Moves too chaotically.
 - Does not exploit promising regions enough.
 - Requires much slower cooling to perform well.
- SA found a physically valid gear set, but not even close to the desired ratio.

Conclusion

PSO is clearly the superior method for this task. It finds a nearly perfect solution extremely fast, while SA remains stuck far away from any optimal region. For a future study, SA could be improved by reducing the cooling rate (e.g., from 0.99 to 0.90) or using a smarter neighbour generator and integer-specific heuristics. But in its current form, SA is not suitable for this optimisation problem.

6 General Conclusion

The investigation into Particle Swarm Optimisation (PSO) and Simulated Annealing (SA) across three engineering domains yields the following conclusions:

- **PSO** demonstrated superior performance in Multi-Objective problems (Truss) and Discrete problems (Gear Train). Its population-based approach allows for better exploration of complex search spaces and effective handling of Pareto optimality.
- **SA** performed well in the continuous, single-objective parameter estimation (Sensor), providing accurate results very efficiently. However, it struggled with the discrete nature of the Gear Train problem without specific tuning of the neighbour generation.

6.1 Performance Comparison

This table summarises the computational performance of the algorithms.

Table 6: Algorithm Performance Metrics

Problem	Algorithm	Run Time (s)	Total Function Calls	Outcome
Two-Bar Truss	PSO (MOPSO)	5.833	20,000	Success (Pareto Front)
Two-Bar Truss	SA	22.858	120,800	Success (Pareto Front)
Sensor Calib.	SA	0.099	3,537	Success (SSE 0.43)
Sensor Calib.	PSO	0.678	25,000	Success (SSE 0.0089)
Gear Train	PSO	0.028	12,000	Success (Error $< 10^{-4}$)
Gear Train	SA	0.0002	88	Failed (High Error)

Note: Function Calls for PSO = Population $\times$ Generations (e.g., $40 \times 300 = 12,000$ for Gear Train). SA Function Calls = Iterations.

6.2 Hyperparameter Sensitivity and Robustness

It is essential to acknowledge that the results presented in this report depend strictly on the specific hyperparameters selected for each algorithm (e.g., the cooling rate α for SA, and the inertia weight w and social/cognitive parameters c_1, c_2 for PSO).

Stochastic optimisation algorithms do not guarantee global convergence, and these settings heavily influence their performance:

- **Simulated Annealing Sensitivity:** In Problem 3 (Gear Train), the SA algorithm failed to converge to the target ratio, resulting in a high error. This failure highlights SA's sensitivity to the cooling schedule and the perturbation step size. A cooling rate that is too fast causes the algorithm to become trapped in local minima, as observed in the Gear Train convergence plot. Conversely, a cooling rate that is too slow results in high computational cost. The failure in the discrete domain suggests that the neighbour-generation method used was not sufficiently tuned to that landscape.
- **PSO Exploration vs. Exploitation:** The PSO performance relies on balancing exploration (controlled by w) and exploitation (controlled by c_1, c_2). While the chosen values ($w = 0.9, c_1 = c_2 = 2.0$) proved robust for the Truss problem, different values could lead to premature convergence or failure to refine the solution.
- **Penalty Factors:** As demonstrated in Section 3.2.1, constrained optimization requires careful tuning of penalty factors. While our experiment showed robustness at $k = 10^{10}$, setting this value too low in other contexts could lead to the acceptance of infeasible solutions, while setting it too high can distort the objective function landscape, making it difficult for the algorithm to approach the feasible boundary.

Therefore, a limitation of this study is that the comparison is valid only for the specific set of parameters tested. A more comprehensive study would require a sensitivity analysis or an automated hyperparameter tuning process (such as a Grid Search) to identify the optimal configuration for each specific problem type.

7 References

- [1] A. Riaz. *Computational Optimisation Design Lecture 1: Introduction to Design Optimisation*. Lecture presented at: Computational Optimisation Design [PowerPoint slides]. Cranfield University. [cited 2025 Oct 20]. Available from: <https://canvas.cranfield.ac.uk/courses/37635/modules>. 2025.
- [2] OpenAI. *ChatGPT [Large language model on the Internet]*. Version GPT-5. San Francisco: OpenAI. [cited 2025 Dec 14]. Available from: <https://chatgpt.com/>. 2025.
- [3] Anthropic. *Claude AI [Large language model on the Internet]*. Version Claude 3.5 Sonnet. San Francisco: Anthropic. [cited 2025 Dec 14]. Available from: <https://claude.ai/>. 2025.
- [4] A. Riaz. *Computational Optimisation Design Lecture 2: Multivariate & Stochastic Optimisation*. Lecture presented at: Computational Optimisation Design [PowerPoint slides]. Cranfield University. [cited 2025 Oct 21]. Available from: <https://canvas.cranfield.ac.uk/courses/37635/modules>. 2025.
- [5] A. Riaz. *Computational Optimisation Design Lecture 3: Multi-objective & Constrained Optimisation*. Lecture presented at: Computational Optimisation Design [PowerPoint slides]. Cranfield University. [cited 2025 Oct 22]. Available from: <https://canvas.cranfield.ac.uk/courses/37635/modules>. 2025.
- [6] S. Kirkpatrick, C.D. Gelatt, and M.P. Vecchi. “Optimization by Simulated Annealing”. In: *Science* (1983).
- [7] G. Chiandussi et al. “Comparison of multi-objective optimization methodologies for engineering applications”. In: *Computers and Mathematics with Applications* 63.5 (Mar. 2012). [cited 2025 Dec 4]. Available from: <https://www.sciencedirect.com/science/article/pii/S0898122111010406>.

8 Appendix

8.1 Multi-objective Particle Swarm Optimisation choice

To implement the Particle Swarm optimisation, a list of criteria with weight is established.

Table 7: Criteria to select the solution for coding Particle Swarm Optimisation

Criterion	Description	Weight
Objective handling capability	Ability to handle both single-objective and multi-objective optimisation problems	6
Method sophistication	Use of advanced optimisation methods rather than basic scalarisation approaches	4
Academic research fit	Suitability and alignment with academic research requirements	1
Algorithm performance	Quality and effectiveness of the optimisation algorithms	3
Swarm-based approach	Use of Particle Swarm optimisation methodology	5
Documentation	Good documentation of the solution	2

GitHub repositories propose solutions but it doesn't respond to the need. The searches are therefore concentrated on libraries.

Table 8: Ranking of possible solutions

Solution		Objective handling capability	Method sophistication	Academic research fit	Algorithm performance	Swarm-based approach	Documentation
Pymoo library	COMPSO method	6	4	1	3	5	2
Pymoo library	MOPSO-CD method	6	4	1	2	5	2
Pymoo library	NSGA-III	6	4	1	3	0	2
Pymoo library	NSGA-II	6	4	1	3	0	2
Pymoo library	R-NSGA-II	6	4	1	3	0	2
Platypus library	NSGA-II	6	4	0	3	0	1
Platypus library	NSGA-II	6	4	0	3	0	1
Platypus library	MOEA/D	6	4	0	3	0	1
Custom MOPSO		6	4	0	2	5	0
PySwarm library	Weighted sum	3	0	0	1	5	2
PySwarm library	Epsilon-constraint	3	2	0	1	5	2

According to the table, the best library is Pymoo and the best algorithm solution is COMPSO method.

8.2 Python code

8.2.1 Two-Bar Plane Truss Problem Definition

Listing 1: Truss Problem Definition

```
" " "  
  
Subject : Two_Bar_Plane_Truss_Problem  
Source : https://www.sciencedirect.com/science/article/pii/  
S0898122111010406  
@author : Clemence-Philomene HINOT & Claude AI  
@date : 04/12/2025  
" " "  
  
# ***** IMPORTATION *****  
  
import math  
import numpy as np  
from pymoo.core.problem import Problem  
  
# ***** CLASS DEFINITION *****  
  
class Two_Bar_Plane_Truss_Problem():  
  
    # ----- INITIALIZATION -----  
  
    def __init__(self):  
        # constants
```

```
self.rho = 7833                # unit: kg/m^3
self.h = 2.54                  # unit: m
self.P = 44482.2               # unit: N
self.E = 2.07e11               # unit: Pa
self.sigma0 = 1.38e8           # unit: Pa
self.A_min = 6.4516e-4         # unit: m^2

# variables
self.pos = 0.0
self.area = 0.0

# physical boundaries
self.pos_min = 0.1 * self.h
self.pos_max = 2.25 * self.h
self.area_min = 0.5 * self.A_min
self.area_max = 2.5 * self.A_min

# ————— CONSTITUTIVE EQUATIONS —————

def f1(self):
    x1 = self.pos/self.h # need to use normalised x1 in this
                          # function
    return 2 * self.rho * self.h * self.area * math.sqrt(1 +
```

pow(x1, 2))

```
def f2(self):  
    x1 = self.pos/self.h  
    return (self.P * pow(self.h, 3) * pow((1 + pow(x1, 2)),  
        1.5)) / ( math.sqrt(1 + pow(x1, 4)) * 2 * math.sqrt(2)  
        * self.E * x1 * self.area)
```

————— *CONSTRAINT EQUATIONS* —————

Stress constraint (tension)

```
def g1(self):  
    x1 = self.pos/self.h  
    return (self.P * (1 + x1) * math.sqrt(1 + pow(x1, 2)) / (2  
        * math.sqrt(2) * x1 * self.area)) - self.sigma0
```

Stress constraint (compression)

```
def g2(self):  
    x1 = self.pos/self.h  
    return (self.P * (1 - x1) * math.sqrt(1 + pow(x1, 2)) / (2  
        * math.sqrt(2) * x1 * self.area)) - self.sigma0
```

————— *EVALUATION FUNCTION* —————

```
def evaluate(self, x):
    """
    Evaluate objectives and constraints

    param x: [x1_normalized, x2_normalized]
    return: ([f1, f2], [g1, g2])
    """

    # physical position
    self.pos = x[0] * self.h

    # physical area
    self.area = x[1] * self.A_min

    # boundaries verifications
    if not (self.pos_min <= self.pos <= self.pos_max and self.
            area_min <= self.area <= self.area_max):
        raise ValueError(f'x values outside boundaries: {
            position={self.pos}, area={self.area}')

    # weight
    f1 = self.f1()
```

```
# movement

f2 = self.f2()

#constraints

g1 = self.g1()
g2 = self.g2()

# results

return [f1 , f2] , [g1 , g2]

# ***** PYMOO WRAPPER *****

class Two_Bar_Plane_Truss_Problem_Pymoo(Problem):

    """Pymoo-compatible wrapper for Two_Bar_Plane_Truss_Problem"""

    def __init__(self , penalty_factor=0):

        # Create truss problem instance

        self.truss = Two_Bar_Plane_Truss_Problem()

        # access to the penalty factor

        self.penalty_factor = penalty_factor
```

```
# Initialize pymoo Problem

super().__init__(
    n_var=2,                                # x1, x2
    n_obj=2,                                # f1, f2
    n_constr=2,                             # g1, g2
    xl=np.array([0.1, 0.5]),                # lower bounds
    xu=np.array([2.25, 2.5])                # upper bounds
)

def _evaluate(self, x, out, *args, **kwargs):
    """Evaluate multiple solutions"""
    n_solutions = x.shape[0]
    F = np.zeros((n_solutions, 2))
    G = np.zeros((n_solutions, 2))

    for i in range(n_solutions):
        try:
            objectives, constraints = self.truss.evaluate(x[i,
                                                            :])
            F[i, :] = objectives
            G[i, :] = constraints
        except ValueError:
            # Penalty for invalid solutions
```



```
F[i, :] = [1e10, 1e10]
```

```
G[i, :] = [1e10, 1e10]
```

```
# output
```

```
out["F"] = F
```

```
out["G"] = G
```

8.2.2 PSO Simulation Script (Truss)

Listing 2: PSO Multi-Objective Simulation

```
"""
Subject: Problem Optimisation with PSO multi-objective
algorithm
@author: Clemence-Philomene HINOT & Claude AI
@date: 04/12/2025
"""

# ***** IMPORTATION *****

import numpy as np
import matplotlib
matplotlib.use('Agg') # Non-interactive backend
import matplotlib.pyplot as plt
import csv
import os
```

```
# PSO algorithms

from Particle_Swarm_Optimization_multi_obj import ParticleSwarm


# problems

from two_bar_plane_truss_problem import
    Two_Bar_Plane_Truss_Problem_Pymoo


# ===== VISUALIZATION FUNCTIONS =====


def plot_pareto_front(result , save_path , obj_labels=None):
    """
    Plot the Pareto front for bi-objective optimization

    :param result: Optimization result from pymoo
    :param save_path: Path to save the figure
    :param obj_labels: List of [xlabel, ylabel] for objectives
    """
    if obj_labels is None:
        obj_labels = ['Objective_1', 'Objective_2']

    fig , ax = plt.subplots(figsize=(10, 6))
    ax.scatter(result.F[:, 0], result.F[:, 1], color='blue', s=50,
```

```
        alpha=0.6, edgecolors='black')
ax.set_xlabel(obj_labels[0], fontsize=12)
ax.set_ylabel(obj_labels[1], fontsize=12)
ax.set_title('Pareto_Front', fontsize=14, fontweight='bold')
ax.grid(True, alpha=0.3)
fig.savefig(save_path, dpi=300, bbox_inches='tight')
plt.close(fig)

print(f"Pareto_front_saved_to_{save_path}")

def plot_decision_space(result, save_path, var_labels=None):
    """
    Plot the decision space colored by first objective

    :param result: Optimization result from pymoo
    :param save_path: Path to save the figure
    :param var_labels: List of [xlabel, ylabel] for decision
        variables
    """
    if var_labels is None:
        var_labels = ['Variable_1', 'Variable_2']

    fig, ax = plt.subplots(figsize=(10, 6))
```

```
scatter = ax.scatter(result.X[:, 0], result.X[:, 1], c=result.
    F[:, 0],
                        cmap='viridis', s=50, alpha=0.6,
                        edgecolors='black')
ax.set_xlabel(var_labels[0], fontsize=12)
ax.set_ylabel(var_labels[1], fontsize=12)
ax.set_title('Decision_Space', fontsize=14, fontweight='bold')
ax.grid(True, alpha=0.3)
fig.colorbar(scatter, label='First_Objective')
fig.savefig(save_path, dpi=300, bbox_inches='tight')
plt.close(fig)
print(f"Decision_Space_saved_to_{save_path}")

def plot_convergence(result, save_path, obj_labels=None):
    """
    Plot_comprehensive_convergence_history

    :param result: Optimization_result_from_pymoo
    :param save_path: Path_to_save_the_figure
    :param obj_labels: List_of_objective_names
    """
    if obj_labels is None:
```

```
obj_labels = ['Objective_1', 'Objective_2']

# Check if callback data exists

if not hasattr(result, 'callback') or result.callback is None:
    print("ERROR: Convergence data not available - callback was not properly initialized")
    return

data = result.callback.data

if "best_f1" not in data or len(data["best_f1"]) == 0:
    print("ERROR: No convergence data recorded")
    return

generations = np.arange(len(data["best_f1"]))

# Create figure with subplots
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Convergence Analysis', fontsize=16, fontweight='bold')

# Plot 1: Best values for each objective
ax1 = axes[0, 0]
```

```
ax1.plot(generations, data["best_f1"], 'b-o', label=obj_labels
        [0], linewidth=2, markersize=4)
ax1.set_xlabel('Generation', fontsize=11)
ax1.set_ylabel(obj_labels[0], fontsize=11, color='b')
ax1.tick_params(axis='y', labelcolor='b')
ax1.grid(True, alpha=0.3)
ax1.set_title('Best Value - First Objective', fontsize=12,
             fontweight='bold')
```

Plot 2: Second objective

```
ax2 = axes[0, 1]
ax2.plot(generations, data["best_f2"], 'r-s', label=obj_labels
        [1], linewidth=2, markersize=4)
ax2.set_xlabel('Generation', fontsize=11)
ax2.set_ylabel(obj_labels[1], fontsize=11, color='r')
ax2.tick_params(axis='y', labelcolor='r')
ax2.grid(True, alpha=0.3)
ax2.set_title('Best Value - Second Objective', fontsize=12,
             fontweight='bold')
```

Plot 3: Both objectives on same plot (normalized)

```
ax3 = axes[1, 0]
f1_norm = (data["best_f1"] - np.min(data["best_f1"])) / (np.
```

```
max(data["best_f1"]) - np.min(data["best_f1"]) + 1e-10)
f2_norm = (data["best_f2"] - np.min(data["best_f2"])) / (np.
max(data["best_f2"]) - np.min(data["best_f2"]) + 1e-10)
ax3.plot(generations, f1_norm, 'b-o', label=obj_labels[0],
linewidth=2, markersize=4)
ax3.plot(generations, f2_norm, 'r-s', label=obj_labels[1],
linewidth=2, markersize=4)
ax3.set_xlabel('Generation', fontsize=11)
ax3.set_ylabel('Normalized_Best_Value', fontsize=11)
ax3.legend(loc='best', fontsize=10)
ax3.grid(True, alpha=0.3)
ax3.set_title('Normalized_Convergence_(Both_Objectives)',
fontsize=12, fontweight='bold')

# Plot 4: Number of non-dominated solutions
ax4 = axes[1, 1]
ax4.plot(generations, data["n_nds"], 'g-^', linewidth=2,
markersize=4)
ax4.set_xlabel('Generation', fontsize=11)
ax4.set_ylabel('Number_of_Solutions', fontsize=11)
ax4.grid(True, alpha=0.3)
ax4.set_title('Pareto_Front_Size_Evolution', fontsize=12,
fontweight='bold')
```

```
plt.tight_layout()

fig.savefig(save_path, dpi=300, bbox_inches='tight')

plt.close(fig)

print(f"Convergence plot saved to {save_path}")


def plot_pareto_evolution(result, save_path, obj_labels=None,
    generations_to_plot=None):
    """
    Plot evolution of Pareto front over generations

    :param result: Optimization result from pymoo
    :param save_path: Path to save the figure
    :param obj_labels: List of objective names
    :param generations_to_plot: List of generation indices to plot
        (default: evenly spaced)
    """
    if obj_labels is None:
        obj_labels = ['Objective_1', 'Objective_2']

    if not hasattr(result, 'callback') or result.callback is None:
        print("ERROR: Cannot plot Pareto evolution - callback data
            not available")
```



```
return

data = result.callback.data
F_history = data["F"]
n_gens = len(F_history)

if generations_to_plot is None:
    # Plot 6 evenly spaced generations
    indices = [0, n_gens//5, 2*n_gens//5, 3*n_gens//5, 4*
               n_gens//5, n_gens-1]
    generations_to_plot = [i for i in indices if i < n_gens]

fig, ax = plt.subplots(figsize=(12, 8))
colors = plt.cm.viridis(np.linspace(0, 1, len(
    generations_to_plot)))

for idx, gen in enumerate(generations_to_plot):
    F = F_history[gen]
    ax.scatter(F[:, 0], F[:, 1], color=colors[idx], s=40,
              alpha=0.6,
              label=f'Gen_{gen}', edgecolors='black',
              linewidth=0.5)
```

```
ax.set_xlabel(obj_labels[0], fontsize=12)
ax.set_ylabel(obj_labels[1], fontsize=12)
ax.set_title('Pareto_Front_Evolution_Across_Generations',
             fontsize=14, fontweight='bold')
ax.legend(loc='best', fontsize=10)
ax.grid(True, alpha=0.3)

fig.savefig(save_path, dpi=300, bbox_inches='tight')
plt.close(fig)
print(f"_Pareto_evolution_plot_saved_to_{save_path}")

# ===== EXPORT FUNCTIONS =====

def save_results_to_csv(result, save_path, var_names=None,
                       obj_names=None):
    """
    Save optimization results to CSV

    param result: Optimization result from pymoo
    param save_path: Path to save CSV file
    param var_names: List of variable names
    param obj_names: List of objective names
    """
```

```
n_var = result.X.shape[1]
n_obj = result.F.shape[1]

if var_names is None:
    var_names = [f'x{i+1}' for i in range(n_var)]
if obj_names is None:
    obj_names = [f'f{i+1}' for i in range(n_obj)]

with open(save_path, 'w', newline='') as f:
    writer = csv.writer(f)

    # Header
    header = var_names + obj_names
    if hasattr(result, 'G') and result.G is not None:
        n_constr = result.G.shape[1]
        header += [f'g{i+1}' for i in range(n_constr)]
    writer.writerow(header)

    # Data
    for i in range(len(result.F)):
        row = list(result.X[i, :]) + list(result.F[i, :])
        if hasattr(result, 'G') and result.G is not None:
            row += list(result.G[i, :])
```

```
writer.writerow(row)

print(f"Results saved to {save_path}")

def save_convergence_to_csv(result, save_path, obj_names=None):
    """
    Save convergence data to CSV

    :param result: Optimization result from pymoo
    :param save_path: Path to save CSV file
    :param obj_names: List of objective names
    """
    if not hasattr(result, 'callback') or result.callback is None:
        print("WARNING: No convergence data to save")
        return

    data = result.callback.data

    if obj_names is None:
        obj_names = ['Objective_1', 'Objective_2']

    with open(save_path, 'w', newline='') as f:
        writer = csv.writer(f)
```

```
# Header

header = [ 'Generation ', f'Best_{obj_names[0]} ', f'Best_{
    obj_names[1]} ', 'Pareto_Front_Size ' ]
writer.writerow(header)


# Data

for i in range(len(data["best_f1"])):
    row = [ i, data["best_f1"][i], data["best_f2"][i], data
        ["n_nds"][i] ]
    writer.writerow(row)


print(f"Convergence_data_saved_to_{save_path}")


def save_summary(result, save_path, var_names=None, obj_names=None
):
    """
    Save summary of best solutions to CSV

    param result : Optimization result from pymoo
    param save_path : Path to save summary CSV
    param var_names : List of variable names
    param obj_names : List of objective names
```

```
#####  
  
n_var = result.X.shape[1]  
n_obj = result.F.shape[1]  
  
if var_names is None:  
    var_names = [f'x{i+1}' for i in range(n_var)]  
if obj_names is None:  
    obj_names = [f'f{i+1}' for i in range(n_obj)]  
  
# Find key solutions  
min_indices = [np.argmin(result.F[:, i]) for i in range(n_obj)]  
]  
middle_idx = len(result.F) // 2  
  
with open(save_path, 'w', newline='') as f:  
    writer = csv.writer(f)  
  
# Header  
writer.writerow(['Summary_of_Key_Solutions'])  
writer.writerow(['Solution_Type'] + var_names + obj_names)  
  
# Min solutions for each objective  
for i, idx in enumerate(min_indices):
```

```
        row = [f'Min_{obj_names[i]}'] + list(result.X[idx, :])
            + list(result.F[idx, :])
        writer.writerow(row)

    # Compromise solution
    row = ['Compromise'] + list(result.X[middle_idx, :]) +
        list(result.F[middle_idx, :])
    writer.writerow(row)

    print(f"Summary saved to {save_path}")

# ===== MAIN OPTIMIZATION RUNNER =====

def run_optimization(problem, pso_params, output_dir='results',
                     var_names=None, obj_names=None,
                     var_labels=None, obj_labels=None):
    """
    Run PSO optimization on a given problem

    param problem : Pymoo Problem instance
    param pso_params : Dictionary of PSO parameters
    param output_dir : Directory to save results
```

```
##### param_var_names: List of variable names for CSV
##### param_obj_names: List of objective names for CSV
##### param_var_labels: List of variable labels for plots
##### param_obj_labels: List of objective labels for plots
##### return: Optimization result
##### " " "

print( "="*70)
print( "MULTI-OBJECTIVE_OPTIMIZATION_USING_PSO")
print( "="*70)

# Create output directory
os.makedirs(output_dir, exist_ok=True)

# Initialize PSO
print( "\n[1/3] Initializing PSO optimizer ... ")
pso = ParticleSwarm(problem, pso_params)
print( " PSO initialized ")

# Run optimization
print( "\n[2/3] Running optimization ... ")
result = pso.PSO_multi_objective(Verbose=True)
print( f" Optimization complete - Found {len(result.F)} Pareto "
```



```
solutions")

# Save results

print("\n[3/3] Saving results ...")

# CSV files

csv_path = os.path.join(output_dir, 'pareto_solutions.csv')
save_results_to_csv(result, csv_path, var_names, obj_names)

summary_path = os.path.join(output_dir, 'summary.csv')
save_summary(result, summary_path, var_names, obj_names)

convergence_csv_path = os.path.join(output_dir, '
    convergence_data.csv')
save_convergence_to_csv(result, convergence_csv_path,
    obj_names)

# Plots

pareto_path = os.path.join(output_dir, 'pareto_front.png')
plot_pareto_front(result, pareto_path, obj_labels)

decision_path = os.path.join(output_dir, 'decision_space.png')
plot_decision_space(result, decision_path, var_labels)
```

```
convergence_path = os.path.join(output_dir, 'convergence.png')  
plot_convergence(result, convergence_path, obj_labels)
```

```
evolution_path = os.path.join(output_dir, 'pareto_evolution.  
    png')  
plot_pareto_evolution(result, evolution_path, obj_labels)
```

```
print("\n" + "="*70)  
print("OPTIMIZATION_COMPLETE!")  
print("="*70)
```

```
return result
```

```
# ===== PROBLEM OPTIMISATION =====
```

```
if __name__ == "__main__":  
    from two_bar_plane_truss_problem import  
        Two_Bar_Plane_Truss_Problem_Pymoo  
  
    # Create problem  
    problem = Two_Bar_Plane_Truss_Problem_Pymoo()
```

```
# Configure PSO parameters
```

```
pso_params = {  
    'pop_size': 100,  
    'w': 0.9,  
    'c1': 2.0,  
    'c2': 2.0,  
    'max_velocity_rate': 0.2,  
    'n_gen': 200,  
    'seed': 42  
}
```

```
# Define labels for visualization and export
```

```
var_names = ['x1', 'x2']  
obj_names = ['Weight', 'Displacement']  
var_labels = ['x1_(normalized_position)', 'x2_(normalized_area  
    )']  
obj_labels = ['Weight_(kg)', 'Displacement_(m)']
```

```
# Run optimization
```

```
result = run_optimization(  
    problem=problem,  
    pso_params=pso_params,
```

```
        output_dir='results_truss ',  
        var_names=var_names ,  
        obj_names=obj_names ,  
        var_labels=var_labels ,  
        obj_labels=obj_labels  
    )
```

8.2.3 Steinhart-Hart Problem Definition

Listing 3: Steinhart-Hart Problem

```
##@Author: John.H  
#Date: 2025-12-03  
#Version: 1.0  
#Description: Steinhart-Hart Problem  
  
import math  
import random  
import csv  
from problem_interface import Problem  
  
class SteinhartHartProblem(Problem):  
    def __init__(self, data_file):  
        self.data = []  
        self._load_data(data_file)
```

```
def _load_data(self, filename):
    """Loads thermistor data from CSV."""
    with open(filename, 'r') as f:
        reader = csv.DictReader(f)
        for row in reader:
            # Store R and T (in Kelvin)
            #  $T_K = T_C + 273.15$ 
            r = float(row['Resistance_Ohms'])
            t_c = float(row['Temperature_C'])
            t_k = t_c + 273.15
            self.data.append({'R': r, 'T_K': t_k})

def generate_initial_solution(self):
    """
    Returns a random starting state [A, B, C].
    Typical values:  $A \sim 1e-3$ ,  $B \sim 1e-4$ ,  $C \sim 1e-7$ 
    """
    return [
        random.uniform(0.0005, 0.002),    # A
        random.uniform(0.0001, 0.0005),   # B
        random.uniform(1e-8, 1e-6)         # C
    ]
```

```
def evaluate(self, solution):
    """
    Calculates SSE:  $\sum (T_{\text{measured}} - T_{\text{model}})^2$ 
    Model:  $\frac{1}{T} = A + B \ln(R) + C (\ln(R))^3$ 
     $\Rightarrow T_{\text{model}} = \frac{1}{A + B \ln(R) + C (\ln(R))^3}$ 
    """
    A, B, C = solution

    sse = 0.0

    for point in self.data:
        R = point['R']
        T_measured = point['T_K']

        try:
            ln_R = math.log(R)
            denom = A + B * ln_R + C * (ln_R ** 3)

            if denom == 0:
                return float('inf')

            T_model = 1.0 / denom
```

```
        error = T_measured - T_model

        sse += error ** 2

    except (ValueError, ZeroDivisionError):

        return float('inf')

    return sse

def get_neighbor(self, solution):
    """
    Perturbs [A, B, C] slightly.
    """
    A, B, C = solution

    # Perturbation factors tailored to the scale of parameters
    new_A = A + random.gauss(0, 1e-5)
    new_B = B + random.gauss(0, 1e-6)
    new_C = C + random.gauss(0, 1e-9)

    return [new_A, new_B, new_C]

def plot_model(self, solution, save_path=None):
    """
    Plots the measured data vs the model curve.
```

```
##### " " "

import matplotlib.pyplot as plt

import numpy as np

A, B, C = solution

# Extract data
R_data = [d['R'] for d in self.data]
T_data = [d['T_K'] - 273.15 for d in self.data] # Convert
        back to Celsius for plotting

# Generate smooth curve
min_R = min(R_data)
max_R = max(R_data)
R_smooth = np.linspace(min_R, max_R, 100)
T_smooth = []

for r in R_smooth:
    try:
        ln_R = math.log(r)
        denom = A + B * ln_R + C * (ln_R ** 3)
        if denom != 0:
            t_k = 1.0 / denom
```



```
T_smooth.append(t_k - 273.15)

else:
    T_smooth.append(None)

except ValueError:
    T_smooth.append(None)

plt.figure(figsize=(10, 6))
plt.scatter(R_data, T_data, color='red', label='Measured_
Data')
plt.plot(R_smooth, T_smooth, color='blue', label='
Steinhart-Hart_Model')

plt.xlabel('Resistance_(Ohms)')
plt.ylabel('Temperature_(Celsius)')
plt.title('Thermistor_Calibration:_Measured_vs_Model')
plt.legend()
plt.grid(True)

if save_path:
    plt.savefig(save_path)
    print(f"Plot_saved_to_{save_path}")

plt.show()
```

8.2.4 Simulated Annealing Class

Listing 4: Simulated Annealing Implementation

```
# Objective: Simulated Annealing Optimization
# Author: John Hoarau
# Date: 2025-12-03

import math
import random
import matplotlib.pyplot as plt
import csv

class SimulatedAnnealing:
    def __init__(self, problem, initial_temp=1000, cooling_rate
                =0.9, min_temp=0.1, seed=None):
        """
        :param seed: Integer to fix randomness for reproducibility
        """
        self.problem = problem
        self.initial_temp = initial_temp
        self.temp = initial_temp
        self.cooling_rate = cooling_rate
```

```
self.min_temp = min_temp

# 1. Set the Seed

if seed is not None:
    random.seed(seed)
    print(f"Random_Seed_set_to:{seed}")

# Data Logging
self.history = {
    "iteration": [],
    "temperature": [],
    "energy": [],
    "best_energy": [],
    "accepted": [] # 1 if accepted, 0 if rejected
}

self.best_solution = None


def _update_temperature(self):
    """
    Implements the Cooling_Schedule.
    Currently: Geometric cooling.
    """
    self.temp *= self.cooling_rate
```

```
def solve(self , verbose=True):  
    current_solution = self.problem.generate_initial_solution  
        (  
      
    current_energy = self.problem.evaluate(current_solution)  
  
    best_solution = current_solution  
    best_energy = current_energy  
  
    iteration = 0  
  
    print("——_Starting_Optimization_——")  
  
    while self.temp > self.min_temp:  
        # Create neighbor  
        new_solution = self.problem.get_neighbor(  
            current_solution)  
        new_energy = self.problem.evaluate(new_solution)  
  
        energy_diff = new_energy - current_energy  
        accepted = False  
  
        # Acceptance Logic
```

```
    if new_energy < current_energy:
        accepted = True
    else:
        probability = math.exp(-energy_diff / self.temp)
        if random.random() < probability:
            accepted = True

# Update State
if accepted:
    current_solution = new_solution
    current_energy = new_energy
    if current_energy < best_energy:
        best_energy = current_energy
        best_solution = current_solution

# Store best solution in instance
self.best_solution = best_solution

# 2. Collect Metrics
self.history["iteration"].append(iteration)
self.history["temperature"].append(self.temp)
self.history["energy"].append(current_energy)
self.history["best_energy"].append(best_energy)
```

```
        self.history["accepted"].append(1 if accepted else 0)

        # Cool down

        self._update_temperature()

        iteration += 1

    print(f"——_Finished_in_{iteration}_iterations_——")
    print(f"Final_Energy:_{best_energy:.4f}")

    # Calculate final Acceptance Ratio

    total_accepted = sum(self.history["accepted"])
    ratio = total_accepted / iteration
    print(f"Global_Acceptance_Ratio:_{ratio:.2%}")

    return best_solution

def plot_statistics(self, save_path=None):

    """
    Generates 2 graphs:
    1. Energy & Temperature over time.
    2. Acceptance Ratio over time.
    """
```

```
iterations = self.history["iteration"]

# Setup the plot
fig, (ax1, ax3) = plt.subplots(2, 1, figsize=(10, 8),
                                sharex=True)

# — Graph 1: Energy vs Temperature —
color = 'tab:red'
ax1.set_ylabel('Energy (Cost)', color=color)
ax1.plot(iterations, self.history["energy"], color=color,
         alpha=0.3, label="Current Energy")
ax1.plot(iterations, self.history["best_energy"], color='
         black', linewidth=2, label="Best Energy")
ax1.tick_params(axis='y', labelcolor=color)
ax1.legend(loc='upper right')

# Create a second y-axis for Temperature sharing the same
# x-axis
ax2 = ax1.twinx()
color = 'tab:blue'
ax2.set_ylabel('Temperature', color=color)
ax2.plot(iterations, self.history["temperature"], color=
         color, linestyle='—', label="Temperature")
```

```
ax2.tick_params(axis='y', labelcolor=color)
ax1.set_title('Optimization_History: Energy & Temperature'
)

# — Graph 2: Cumulative Acceptance Ratio —
# Calculate running average of acceptance
cumulative_acceptance = []
running_sum = 0
for i, val in enumerate(self.history["accepted"]):
    running_sum += val
    cumulative_acceptance.append(running_sum / (i + 1))

ax3.set_xlabel('Iterations')
ax3.set_ylabel('Acceptance_Ratio')
ax3.plot(iterations, cumulative_acceptance, color='green',
        label="Cumulative_Acceptance_Ratio")
ax3.axhline(y=0.0, color='r', linestyle='-', alpha=0.3)
ax3.set_title('Cooling_Efficiency_(Acceptance_Ratio)')
ax3.grid(True)

plt.tight_layout()

if save_path:
```



```
plt.savefig(save_path)

print(f"Plot saved to {save_path}")

plt.show()

def plot_best_solution(self, save_path=None):
    """
    Plots the best solution found by the optimizer.
    """
    if self.best_solution is not None:
        self.problem.plot_model(self.best_solution, save_path=
                                save_path)
    else:
        print("No solution found yet. Run solve() first.")

def plot_model(self, solution, save_path=None):
    """
    Plots a specific solution.
    """
    self.problem.plot_model(solution, save_path=save_path)

def save_to_csv(self, filename="annealing_results.csv"):
    """
```

```
##### Saves the history of the run to a CSV file .  
##### Format: iteration , temperature , energy , best_energy ,  
        accepted  
##### " " "
```

```
# 1. Get the column names (headers)  
keys = list(self.history.keys())  
  
# 2. Get the data columns  
values = list(self.history.values())  
  
# 3. Zip them together (this turns columns into rows)  
rows = zip(*values)  
  
try:  
    with open(filename, 'w', newline='') as f:  
        writer = csv.writer(f)  
  
        # Write the header  
        writer.writerow(keys)  
  
        # Write the data  
        writer.writerows(rows)
```

```
print(f"→Data successfully saved to '{filename}')
```

```
except IOError as e:
```

```
print(f"Error saving file: {e}")
```

8.2.5 Gear Train Problem and Runner